ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

Construindo Aplicativos Móveis com React Native

@profalexandrejr

Ementa

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

- **Aula 01:** Nivelamento - O JavaScript Moderno (ES6+)
- **Aula 02:** Introdução ao React Native com Expo
- **Aula 03:** Componentização e UI Profissional
- **Aula 04:** Interatividade e Estado (Hooks Parte 1)
- **Aula 05:** Listas Otimizadas e Scroll
- **Aula 06:** Ciclo de Vida e Efeitos (Hooks Parte 2)
- **Aula 07:** Navegação I - Stack Navigation
- **Aula 08:** Navegação II - Menus e Tabs
- **Aula 09:** Armazenamento Local
- **Aula 10:** Integração com APIs

@profalexandrejr

Configuração do Ambiente

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

Clique na imagem para ser redirecionado ao link oficial de download da aplicação.

Visual Studio Code

Expo Go

iOS

POSTMAN

@profalexandrejr

Aula 1

Nivelamento:
O JavaScript Moderno (ES6+)

Variáveis e Escopo

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

- Por que abandonar o **var**?
  - O **var** ignora blocos de código (**if**, **for**). Ele "vaza" escopo, o que causa *bugs* silenciosos.
  - *Dica*: Esqueça o **var**. No React Native, não o usamos.
- Diferença entre **let** (mutável) e **const** (imutável):
  - **let** (Escopo de Bloco)
    - Respeita as chaves **{ }**. A variável só existe ali dentro.
    - Permite reatribuição (mudar o valor da variável).
  - **const** (A Regra de Ouro)
    - Não permite reatribuição.
    - *Dica de Ouro*: Comece sempre com **const**. Se precisar mudar o valor depois, troque para **let**.

Variáveis e Escopo

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

JavaScript

```javascript
// ❌ O jeito antigo (var vaza)
if (true) {
  var nome = "Alexandre";
}
console.log(nome); // Funciona (mas não deveria!)

// ✅ O jeito seguro (let/const)
if (true) {
  let idade = 30;
  const cpf = "123.456.789-00";
  // cpf = "000..."; // ERRO! Const não muda.
}
// console.log(idade); // ERRO! Variável não existe aqui fora.
```

@profalexandrejr

Arrow Functions

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

- Sintaxe tradicional vs. Arrow Function:
  - Removemos a palavra **function** e usamos a "seta" (*arrow*) **=>**.
- Retornos implícitos (limpeza de código):
  - Se a função tem apenas uma linha, podemos remover as chaves **{ }** e a palavra **return**.
  - *Por que isso importa?* Muito usado em filtros e *loops* dentro do JSX.

Arrow Functions

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

JavaScript

```javascript
// 🐢 Tradicional
function somar(a, b) {
  return a + b;
}

// 🚀 Arrow Function (Com corpo)
const somar = (a, b) => {
  return a + b;
};

// 🔥 Arrow Function (Retorno Implícito - One-liner)
// "Recebe A e B e retorna A + B"
const somar = (a, b) => a + b;
```

@profalexandrejr

Operações com Arrays

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

- **map()**: Transformando dados.
  - Percorre a lista e cria uma nova lista do mesmo tamanho, mas transformada.
  - *Uso no React*: Transformar uma lista de dados (JSON) em uma lista de componentes visuais.
- **filter()**: Selecionando dados.
  - Cria uma nova lista apenas com os itens que passaram no teste (retornaram **true**).
  - *Uso no React*: Remover um item de uma lista de tarefas.
- **find()**: Buscando um item.
  - Retorna **o primeiro item** que satisfaz a condição.

Operações com Arrays

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

JavaScript

```javascript
const precos = [10, 20, 35, 50];

// MAP: Aplica 10% de desconto em todos
const descontos = precos.map(item => item * 0.9);
// Resultado: [9, 18, 31.5, 45]

// FILTER: Apenas preços menores que 30
const baratos = precos.filter(item => item < 30);
// Resultado: [10, 20]

// FIND: Achar exatamente o preço 50
const alvo = precos.find(item => item === 50);
// Resultado: 50
```

@profalexandrejr

Desestruturação e Spread Operator

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

- Desestruturação (*Destructuring*)
  - Extrai dados de dentro de um objeto diretamente para variáveis (**const { nome } = usuario**).
  - Evita repetição de código (**usuario.nome**, **usuario.email**, **usuario.idade**...).
- *Spread Operator* **...** (Espalhar)
  - Copia todas as propriedades de um objeto para outro.
  - Essencial para **Imutabilidade** (criar uma cópia atualizada sem mudar o original).

Desestruturação e Spread Operator

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

JavaScript

```javascript
const usuario = { nome: "Ana", idade: 25, cidade: "Recife" };

// 📦 Desestruturação
// Em vez de: const nome = usuario.nome;
const { nome, cidade } = usuario;
console.log(nome); // "Ana"

// ✨ Spread Operator (Atualizando dados)
// "Crie um novo objeto com tudo de 'usuario', mas mude a cidade"
const usuarioAtualizado = { ...usuario, cidade: "São Paulo" };

console.log(usuarioAtualizado);
// { nome: "Ana", idade: 25, cidade: "São Paulo" }
```

@profalexandrejr

Módulos (Import/Export)

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

- Organização de arquivos:
  - **export default** (O Principal)
    - Só pode haver **um** por arquivo.
    - Na importação, você pode dar **qualquer nome**.
    - *Uso*: Geralmente usado para o Componente principal da tela.
  - **export** (Nomeado)
    - Podem haver vários por arquivo.
    - Na importação, o nome deve ser exato e estar entre chaves **{ }**.
    - *Uso*: Funções utilitárias, estilos, constantes.

Módulos (Import/Export)

ALEXANDRE JUNIOR
PROFESSOR E DESENVOLVEDOR

JavaScript

```javascript
// --- Arquivo: uteis.js ---
export const PI = 3.14; // Export Nomeado
export default function calcularArea() { ... } // Default

// --- Arquivo: App.js (Como importar) ---
import MinhaFuncao, { PI } from './uteis';

// Note que 'calcularArea' virou 'MinhaFuncao' (eu escolho o nome)
// Mas 'PI' teve que ser importado com o nome exato entre { }
```

Obrigado!

@profalexandrejr